Degree-preserving spanning trees on strongly chordal graphs and directed path graphs

Ching-Chi Lin[‡] Gerard J. Chang[†*] Gen-Huey Chen[‡]

[‡]Department of Computer Science and Information Engineering
National Taiwan University, Taipei, Taiwan
Email:{d91018,ghchen}@csie.ntu.edu.tw

[†]Department of Mathematics
National Taiwan University, Taipei, Taiwan
Email: gjchang@math.ntu.edu.tw

Abstract

Suppose G is a connected graph and T a spanning tree of G. A vertex $v \in V(G)$ is said to be a degree-preserving vertex if its degree in T is the same as in G. The degree-preserving spanning tree problem is to find a spanning tree T of a connected graph G such that the number of degree-preserving vertices is maximum. The purpose of this paper is to give an $O(m\alpha(m,n))$-time algorithm for the degree-preserving spanning tree problem on strongly chordal graphs and an $O(m+n)$-time algorithm on directed path graphs, where α is the inverse of Ackermann's function.

1 Introduction

Suppose G is a connected graph and T is a spanning tree of G. A vertex $v \in V(G)$ is said to be a *degree-preserving vertex* if its degree in T is the same as in G. The *degree-preserving spanning tree* (DPST) problem is to find a spanning tree T of a connected graph G such that the number of degree-preserving vertices is maximum. Finding degree-preserving trees is not only fundamental in algorithmic graph theory [2, 3, 5, 10] but also practically important in designing water-distribution networks.

Water-distribution networks are often modelled as graphs G. The vertices in $V(G)$ represent gates in the network and the edges in $E(G)$ represent pipes. When measuring the flows in all edges, it is a natural way to install flow-meters on all the edges. However, with flow conservation, i.e., the incoming flow is the same as the outgoing flow for a vertex except the source vertex and the sink vertex, we can reason the flow on edges of the spanning tree of G to reduce the cost of flow-meters on edges the spanning tree. Since the pressure gauges is much more inexpensive than the flow meters, installing pressure gauges on the vertices would be cost-effective.

The flow on an edge can be computed by measuring the water-pressures on the two incident vertices. Notice that flow conservation still can be applied. So, only the flows of edges of the cotree should be measured, i.e., a *cotree* of G is obtained from G by deleting the edges of a spanning tree of G. In order to reduce the number of pressure gauges used on vertices, we would like to find a cotree whose edges are incident with a minimum number of vertices. Alternatively, the problem could be defined as finding a spanning tree T such

*Supported in part by the National Science Council under grant NSC-95-2221-E-002-125-MY3. Taida Institute for Mathematical Sciences, National Taiwan University, Taipei 10617, Taiwan. National Center for Theoretical Sciences, Taipei Office.

that the number of degree-preserving vertices is maximum which is the problem of this paper.

Figure 1 shows a graph G with a spanning tree T_1 containing two degree-preserving vertices u and v, and a spanning tree T_2 containing no degree-preserving vertex. The graph G needs installing 11 flow meters on all edges in order to measure the flows in all edges. By using the flow conservation law, it only needs 7 flow meters. Moreover, it needs installing 7 pressure gauges on all vertices if we use pressure gauges to measure the flows. Also, by flow conservation law, it only needs 5 and 7 pressure gauges with respect to the two spanning trees T_1 and T_2, respectively. This example tells us how important the degree-preserving spanning tree is in designing water-distribution networks.

Broersma et al. [3] proved that the DPST problem is NP-complete for bipartite planar degree-6 graphs and split graphs. Furthermore, they also gave a linear-time algorithm for interval graphs, algorithms for graphs with bounded treewidth and graphs with a bounded asteroidal number in polynomial time, an $O(n^4)$-time algorithm for cocomparability graphs, and a linear-time approximation algorithm with ratio $1+\epsilon$ for planar graphs. Bhatia et al. [2] proposed an approximation algorithm for the DPST problem with ratio $\Theta(n^{1/2})$ for general graphs. Khuller et al. [10] improved the result with ratio $2+\epsilon$. Moreover, when the input graph is planar, they also gave a polynomial-time approximation scheme for this problem. Damaschke [5] proved that the DPST problem is NP-complete for bipartite planar degree-5 graphs and for planar degree-3 graphs.

Since split graphs are chordal, the DPST problem is NP-complete for chordal graphs. It is well known that the family of strongly chordal graphs is a proper subfamily of the family of chordal graphs, and is a proper superfamily of the family of interval graphs. Also, the family of circular-arc graphs is a proper superfamily of the family of interval graphs. In this paper, we give an $O(m\alpha(m,n))$-time algorithm for the DPST problem on strongly chordal graphs and an $O(m+n)$-time algorithm on directed path graphs, where α is the inverse of Ackermann's function.

In a graph G, a subset $S \subseteq V(G)$ is *realizable* if there exists a spanning forest F of G such that $d_F(v) = d_G(v)$ for every vertex $v \in S$; in which case, F is called an *S-preserving forest*. A *maximum realizable set* of G is a realizable set whose cardinality is the largest among all realizable sets of G. As showed by Broersma et al. [3], the problem of finding a degree-preserving spanning tree of a connected graph G is equivalent to finding a maximum realizable set of G. In fact, a degree-preserving spanning tree with S as the set of all degree-preserving vertices is an S-preserving forest. On the other hand, an S-preserving forest can be extended to a spanning tree for which all vertices of S are degree-preserving. In order to simplify our algorithms, we shall solve the DPST problem by finding a maximum realizable set S.

The *neighborhood* $N_G(v)$ of a vertex v is the set of all vertices adjacent to v in G; and the *closed neighborhood* $N_G[v] = \{v\} \cup N_G(v)$. For a nonempty subset $S \subseteq V(G)$, let $G[[S]]$ denote the spanning subgraph whose edge set is $\{xy \in E(G) : x \in S\}$. The definition for $G[[S]]$ we give above is slightly different from that in [3], in which the vertex set of $G[[S]]$ is $\cup_{v\in S} N_G[v]$. The difference is that we add isolated vertices to make $G[[S]]$ a spanning subgraph of G for technical purpose. Notice that an edge in $G[[S]]$ has at least one end vertex in S. The following is a useful characterization for a set to be realizable.

Lemma 1 ([3]) *Suppose S is a vertex set of G. Then, S is realizable if and only if $G[[S]]$ is a forest.*

The remainder of the paper is organized as follows. Section 2 describes an $O(m\alpha(m,n))$-time algorithm for the DPST problem in strongly chordal graphs. Section 3 describes and analyzes an $O(m+n)$-time algorithm for the DPST problem in directed path graphs. Section 4 concludes the paper with an open question.

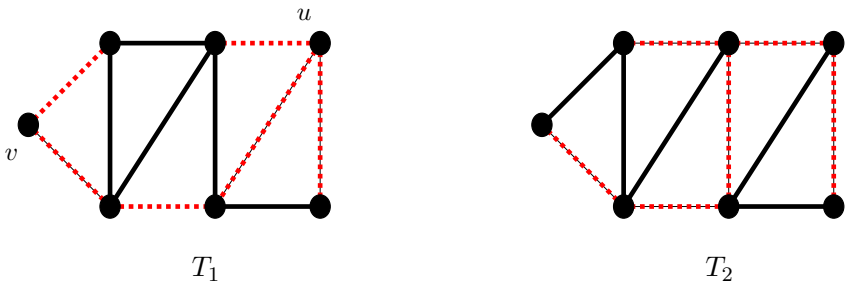

Figure 1: Graph G with spanning trees T_1 and T_2 (dotted edges).

2 Algorithm for strongly chordal graphs

This section establishes an $O(m\alpha(m,n))$-time algorithm for finding a maximum realizable set of a strongly chordal graph. The problem of finding maximum realizable sets on chordal graphs is NP-complete but it is linear-time on interval graphs. The class of strongly chordal graphs is a proper superfamily of interval graphs but a proper subfamily of chordal graphs. Therefore, it is interesting to design an $O(m\alpha(m,n))$-time algorithm for strongly chordal graphs. First, some preliminaries on strongly chordal graphs are introduced.

A graph is *chordal* (or *triangulated*) if every cycle of length greater than three has a chord, which is an edge joining two noncontiguous vertices in the cycle. A vertex v is *simplicial* if $N_G[v]$ is a clique. For any ordering $(v_1, v_2, \ldots, v_n)$ of $V(G)$, let G_i denote the subgraph of G induced by $\{v_i, v_{i+1}, \ldots, v_n\}$. It is well known [8] that a graph G is chordal if and only if it has a *perfect elimination order*, which is an ordering $(v_1, v_2, \ldots, v_n)$ of $V(G)$ such that each v_i is a simplicial vertex of G_i.

A *strongly* (*sun-free*) *chordal* graph is a chordal graph such that every cycle of even length at least six has a chord that divides the cycle into two odd length paths [4]. Farber [7] proved that a graph is strongly chordal if and only if it has a *strong elimination order* which is an ordering $(v_1, v_2, \ldots, v_n)$ of $V(G)$ such that $N_{G_i}[v_j] \subseteq N_{G_i}[v_k]$ for $i \le j \le k$ and $v_j, v_k \in N_{G_i}[v_i]$. Notice that a strong elimination order is also a perfect elimination order. Anstee and Farber [1] presented an $O(n^3)$-time algorithm, Hoffman, Kolen, and Sakarovitch [9] presented an $O(n^3)$-time algorithm, Lubiw [11] presented an $O(m \log^2 m)$-time algorithm, Paige and Tarjan [12] presented an $O(m \log m)$-time algorithm and Spinrad [13] presented an $O(n^2)$-time algorithm for finding a strong elimination order of a strongly chordal graph of n vertices and m edges.

Our algorithm for finding a maximum realizable set of a strongly chordal graph G is a greedy one. Suppose $(v_1, v_2, \ldots, v_n)$ is a strong elimination order of G.

> The algorithm starts with $S = \emptyset$, and processes vertex v_i for i from 1 to n by adding v_i to S whenever $G[[S \cup \{v_i\}]]$ is a forest.

To implement the algorithm efficiently, we employ the data structure of set union and find as follows. At all time we keep a partition of $V(G)$ which corresponds to the trees of $G[[S]]$. There are three operations involved in the data structure. The Make-Set(v) operation creates a new set containing v. The Find-Set(v) operation returns the set s containing v. The Union(s,t) operation unites the two sets s and t. Tarjan [14] gave an worse-case running time $O(\ell\alpha(\ell,k))$ for ℓ disjoint-set operations on k elements, where α is the inverse of Ackermann's function.

By the help of the data structure, we first initialize the set S to be the empty set by doing

Make-Set(v) for each vertex $v \in V(G)$, which creates the n trees of $G[[S]]$ each of one vertex. At each iteration, we may check whenever $G[[S \cup \{v_i\}]]$ is a forest or not by checking if the vertices in $N_G[v_i] - S$ belong to different sets. If the answer is positive, then we add v_i into S and combine the sets which the closest neighborhood of v_i belong to into a new set. Otherwise, we check the next vertex. Since v_i and the neighbors of v_i in $N_G(v_i) \cap S$ are in the same tree and the neighbors of v_i in $N_G(v_i) - S$ belong to different trees when v_i is added into S, $G[[S]]$ is still a forest and so S is realizable. We shall show that S is maximum and the algorithm runs in $O(m\alpha(m,n))$ time in the next theorem.

Algorithm Strongly-Chordal.
Input: A strongly chordal graph G of order n with a strong elimination order $(v_1, v_2, \ldots, v_n)$.
Output: A maximum realizable set S of G.

1. $S \leftarrow \emptyset$
2. **for** each vertex $v \in V(G)$ **do** Make-Set(v)
3. **for** $i = 1$ **to** n **do**
4. **if** Find-Set(v_g) $\neq$ Find-Set(v_h) for each $v_g, v_h \in N_G[v_i] - S$
5. **then** $S \leftarrow S \cup \{v_i\}$
6. Union(Find-Set(v_i),Find-Set(v_j)) for each vertex $v_j \in N_G(v_i) - S$
7. **return** S

Theorem 2 *For a strongly chordal graph G with a strong elimination order provided, Algorithm Strongly-Chordal outputs a maximum realizable set S of G in $O(m\alpha(m,n))$ time.*

Proof. We first prove that $S = \{v_{s_1}, v_{s_2}, \ldots, v_{s_r}\}$, where $s_1 < s_2 < \ldots < s_r$, is a maximum realizable set of G. Choose a maximum realizable set S' of G. If $S = S'$, then we are done. Also, S is not a proper subset of S', for otherwise there is a vertex $v_i \in S' - S$ which must be chosen into S in the greedy algorithm. Hence, $S - S' \neq \emptyset$ and then we may choose the minimum index $p = p(S')$ such that $v_{s_p} \in S - S'$. Without loss of generality, we may assume that the set S' is chosen so that $p(S')$ is maximum. Notice that $\{v_{s_1}, v_{s_2}, \ldots, v_{s_{p-1}}\} \subseteq S \cap S'$. We first give a claim.

Claim A. Suppose $\{v_{s_1}, v_{s_2}, \ldots, v_{s_p}\} \subseteq S'' \subseteq \{v_{s_p}\} \cup S'$. If $G[[S'']]$ has a cycle C, then it has a cycle $C_3 = (v_{s_p}, v_a, v_x)$ with $\{v_a, v_x\} \cap S'' = \{v_x\}$ and $s_p < x$, or else a cycle $C_4 = (v_{s_p}, v_a, v_x, v_b)$ with $\{v_x, v_a, v_b\} \cap S'' = \{v_x\}$ and $s_p < x, a, b$.
Proof of Claim A. Suppose v_c is a minimum indexed vertex of C with two neighbors v_d and v_e in C, where $c < d < e$. First, $v_d v_e \in E(G)$. For the case when either v_d or v_e is in S'', by the definition of $G[[S'']]$, the edge $v_d v_e$ is also in $G[[S'']]$ and so $C_3 = (v_c, v_d, v_e)$ is a cycle in $G[[S'']]$. Notice that $|\{v_c, v_d, v_e\} \cap S''| \geq 2$. It is the case that $|C_3 \cap S| \leq 1$ (respectively, $|C_3 \cap S'| \leq 1$) for otherwise C_3 is a cycle in $G[[S]]$ (respectively, $G[[S']]$), which is impossible. These give that $C_3 = (v_{s_p}, v_a, v_x)$ with $\{v_a, v_x\} \cap S'' = \{v_x\}$. Since $\{v_{s_1}, v_{s_2}, \ldots, v_{s_{p-1}}\} \subseteq S \cap S'$ and $v_{s_p} \in S$, we have $s_p < x$. For the case when neither v_d nor v_e is in S'', the edge $v_d v_e$ is not in $G[[S'']]$ and so we may choose the other neighbor v_f of v_d in C. Then, $v_f \in N_{G_c}[v_d] \subseteq N_{G_c}[v_e]$ and so $v_f v_e \in E(G)$. As $v_d, v_e \notin S''$, we have $v_c, v_f \in S''$ and so $C_4 = (v_c, v_d, v_f, v_e)$ is a cycle in $G[[S'']]$. Again, $|C_4 \cap S| \leq 1$ and $|C_4 \cap S'| \leq 1$ imply that $C_4 = (v_{s_p}, v_a, v_x, v_b)$ with $\{v_x, v_a, v_b\} \cap S'' = \{v_x\}$ and $s_p < x, a, b$. The claim then follows.

Having Claim A in mind, we are ready to prove the theorem. First, $\{v_{s_p}\} \cup S'$ is not a realizable set and so $G[[\{v_{s_p}\} \cup S']]$ has a cycle. By Claim A, it in fact has a cycle (v_{s_p}, v_a, v_x) with $v_x \in S'$ and $s_p < x$, or a cycle (v_{s_p}, v_a, v_x, v_b) with $v_x \in S'$ and $s_p < x, a, b$. For the former case, let $v_k = v_{s_p}$; for the latter case, let $v_k = v_a$. So, there is a vertex $v_x \in S'$ with $s_p < x$ such that there is a vertex $v_k \in N_{G_{s_p}}[v_{s_p}] \cap N_{G_{s_p}}[v_x]$. Without loss of generality, we may assume that the vertices v_x and v_k are chosen so that the index k is minimum.

Next, consider the set $S''' = S' \cup \{v_{s_p}\} - \{v_x\}$. We shall prove that S''' is realizable. Suppose to

the contrary that $G[[S''']]$ has a cycle. By Claim A, it in fact has a cycle $C_3' = (v_{s_p}, v_c, v_y)$ with $v_y \in S' - \{v_x\}$ and $s_p < y$, or a cycle $C_4' = (v_{s_p}, v_c, v_y, v_d)$ with $v_y \in S' - \{v_x\}$ and $s_p < y, c, d$.

For the case when $G[[S''']]$ has a cycle C_3', by the choice of v_x and v_k it is the case that $v_k = v_{s_p}$. Then, $v_x, v_y \in N_{G_{s_p}}[v_{s_p}]$ and so $v_x v_y \in E(G)$. These give a cycle (v_{s_p}, v_x, v_y) in $G[[S']]$ which is impossible.

For the case when $G[[S''']]$ has a cycle C_4', vertices $v_c, v_d \in N_{G_{s_p}}[v_{s_p}] \cap N_{G_{s_p}}[v_y]$. By the choice of v_x and v_k, index $k \leq c, d$. Then, $v_x \in N_{G_{s_p}}[v_k] \subseteq N_{G_{s_p}}[v_c] \cap N_{G_{s_p}}[v_d]$ and so (v_x, v_c, v_y, v_d) is a cycle in $G[[S']]$ which is impossible.

These prove that S''' is realizable. It is in fact a maximum realizable set of G with $p(S''') > p(S')$, contradicting the choice of S'. So, in fact $S = S'$ at the beginning and then S is a maximum realizable set as desired.

Next we explain that the time complexity for the algorithm is $O(m\alpha(m, n))$. The for loop in line 2 performs $O(n)$ Make-Set() operations. Furthermore, the for loop in lines 3–6 checks whether the vertices in $N_G[v_i] - S$ belong to different sets and performs Union() operation if necessary for $i = 1$ to n. It takes $O(|N_G[v_i]|)$ Find-Set() operations to do checking for each vertex v_i. Hence, the for loop in lines 3–6 performs $O(m)$ Find-Set() operations and $O(n)$ Union() operations. Then, Algorithm Strongly-Chordal totally performers $O(m)$ disjoint-set operations on n elements, which takes $O(m\alpha(m, n))$ time. □

3 Algorithm for directed path graphs

This section establishes an $O(n + m)$-time algorithm for finding maximum realizable sets on directed path graphs. Broersma et al. [3] gave a linear-time algorithm for interval graphs. Also, in the previous section, we gave an $O(m \log n)$-time algorithm for strongly chordal graphs. The class of directed path graphs is a proper superfamily of interval graphs and a proper subfamily of strongly chordal graphs. Therefore, it is interesting to design an $O(n+m)$-time algorithm for directed path graphs. First, some preliminaries on directed path graphs are introduced.

A graph is *directed path* if it has an intersection model $(T_G, \mathcal{P})$ consisting of a family $\mathcal{P}$ of directed paths in a rooted directed tree T_G. The vertices in G correspond to directed paths in $\mathcal{P}$ and two vertices in G are adjacent if and only if their corresponding directed paths contain a common vertex. Dietz [6] gave a linear-time algorithm to recognize directed path graphs. As a byproduct, an intersection model $(T_G, \mathcal{P})$ of a directed path graph G can be constructed in linear time.

For a vertex v of G, let $P(v)$ denote the corresponding directed path in $\mathcal{P}$. A directed path $P(v)$ that begins at endpoint $t(v)$ and ends at endpoint $h(v)$ is denoted by $(t(v), h(v))$, where $t(v)$ is the *tail* of $P(v)$ and $h(v)$ is the *head* of $P(v)$. The distance from the root to $t(v)$ is less than the distance from the root to $h(v)$ in T_G. Assume without loss of generality that all endpoints are distinct. For two endpoints x and y in T_G, x is an *ancestor* of y if there is a directed path in T_G from x to y. A *tail order* of G is an ordering $v_1, v_2, \ldots, v_n$ of $V(G)$ such that for $i < j$ the distance from the root to $t(v_i)$ is at least the distance from the root to $t(v_j)$ in T_G.

Lemma 3 *Every tail order $v_1, v_2, \ldots, v_n$ of a directed path graph G is a strong elimination order. Consequently, a directed path graph is strongly chordal.*

Proof. To prove that $v_1, v_2, \ldots, v_n$ is a strong elimination order of G, it suffices to show that $N_{G_i}[v_j] \subseteq N_{G_i}[v_k]$ for $i \leq j \leq k$ and $v_j, v_k \in N_{G_i}[v_i]$. Since v_j and v_k are two closed neighbors of v_i in G_i with $i \leq j \leq k$, it follows from the definition of a tail order that $P(v_j)$ and $P(v_k)$ both contain $t(v_i)$, and $t(v_k)$ is an ancestor of $t(v_j)$. Therefore, $P(v_k)$ contains the path $(t(v_j), t(v_i))$ in T_G, which implies $N_{G_i}[v_j] \subseteq N_{G_i}[v_k]$. □

Our algorithm for finding a maximum realizable set of a directed path graph is essentially the greedy one for strongly chordal graphs. The only difference is the implementation, as now directed path graphs have more structural properties.

Suppose $v_1, v_2, \ldots, v_n$ is a tail order of G. Algorithm Directed-Path first initializes the set S to be the empty set and set $\text{vis}[v] = v$ for each vertex $v \in V(G)$. It then iterates for i from 1 to n to check whether $\text{vis}[v_g] \neq \text{vis}[v_h]$ for each $v_g, v_h \in N_G[v_i] - S$. If positive, then we add v_i into S and set $\text{vis}[v_j] = v_i$ for each $v_j \in N_G(v_i) - S$. Otherwise, we check the next vertex.

This algorithm is quite similar to Algorithm Strongly-Chordal. Instead of using Find-Set() operation to check whether the neighbors of v_i belong to different trees, we check this by testing whether $\text{vis}[v_g] \neq \text{vis}[v_h]$ for each $v_g, v_h \in N_G[v_i] - S$. Also, instead of using Union() operation to combine the sets which the closest neighborhood of v_i belong to into a new set, we only set $\text{vis}[v_j] = v_i$ for each $v_j \in N_G(v_i) - S$ to indicate the closest neighbors of v_i belong to the same tree. Since a tail order of G is also a strong elimination order of G, to prove the correctness of this algorithm, it suffices to show the correctness of the above testing.

For the case when the input graph G is not connected, it is easy to see that the union of maximum realizable sets of the components of G is a maximum realizable set of G. Also, if G is connected with a cut-edge e, then the union of maximum realizable sets of the two components of $G - e$ is a maximum realizable set of G. Therefore, for technical purpose, we may suppose that G is 2-edge connected in this section.

Algorithm Directed-Path.
Input: A 2-edge connected directed path graph G of order n with a tail order $(v_1, v_2, \ldots, v_n)$.
Output: A maximum realizable set S of G.

1. $S \leftarrow \emptyset$
2. **for** each vertex $v \in V(G)$ **do** $\text{vis}[v] \leftarrow v$
3. **for** $i = 1$ to n **do**
4. **if** $\text{vis}[v_g] \neq \text{vis}[v_h]$ for each $v_g, v_h \in N_G[v_i] - S$
5. **then** $S \leftarrow S \cup \{v_i\}$
6. $\text{vis}[v_j] \leftarrow v_i$ for each vertex $v_j \in N_G(v_i) - S$
7. **return** S

Lemma 4 *Suppose v_i is chosen by Algorithm Directed-Path to add into S at the i-th iteration. Let D denote the component of $G[[S]]$ containing v_i.*

1. *If $v_j \in V(D)$ with $j > i$, then v_j is a neighbor of v_i in G.*
2. *If $v_j \in V(D)$ with $j < i$, then $t(v_i)$ is an ancestor of $t(v_j)$ in T_G.*

Proof. Notice that if $v_j \in N_G(v_i)$ with $j > i$, then $v_j \notin S$ at the i-th iteration. Therefore, there exists no directed path $P(v_h)$ containing $t(v_i)$ with $v_h \in S$ and $h > i$ at the i-th iteration. So, if $v_j \in V(D)$ with $j > i$ then v_j must be a neighbor of v_i. Furthermore, if $v_j \in V(D)$ with $j < i$, then $t(v_i)$ must be an ancestor of $t(v_j)$ in T_G. □

Lemma 5 *During the execution of the* **if** *statement at the i-th iteration, any vertex v_k with $k \geq i$ has two neighbors v_g and v_h in $N_G[v_k] - S$ such that $\text{vis}[v_g] = \text{vis}[v_h]$ if and only if v_k has two neighbors in $N_G[v_k] - S$ belonging to the same component D of $G[[S]]$, where $\text{vis}[v_g] \in V(D) \cap S$.*

Proof. Clearly, if v_k has two neighbors v_g and v_h such that $\text{vis}[v_g] = \text{vis}[v_h]$, then v_g and v_h are the two neighbors of v_k belonging to component D, where $\text{vis}[v_g] \in V(D) \cap S$.

We prove the if part by induction on i. The statement holds for $i = 1$ as we have $S = \emptyset$ and $\text{vis}[v] = v$ for each vertex $v \in V(G)$. By the induction hypothesis, if v_k with $k \geq i$ has two neighbors in $N_G[v_k] - S$ belonging to the same component D, then v_k has two neighbors v_g and v_h in $N_G[v_k] - S$ such that $\text{vis}[v_g] = \text{vis}[v_h]$, where $\text{vis}[v_g] \in V(D) \cap S$. If $v_i \notin S$, then the statement

obviously holds for $i+1$. Therefore, we assume $v_i \in S$ in the following discussion.

Consider the case when D does not contain v_i. Notice that we set $\text{vis}[v] = v_i$ for each vertex $v \in N_G[v_i] - S$ at i-th iteration when v_i is added into S. Since $N_G[v_i] \cap V(D) = \emptyset$, $\text{vis}[v]$ does not change for each vertex $v \in V(D)$. So, v_k with $k \geq i+1$ still has two neighbors v_g and v_h such that $\text{vis}[v_g] = \text{vis}[v_h]$, where $\text{vis}[v_g] \in V(D) \cap S$.

Now, consider the the case when D contains v_i. Suppose v_j is a neighbor of v_k in $N_G[v_k] - S$ with $v_j \in V(D)$. By Lemma 4, if $j > i$, then v_j is a neighbor of v_i in G, which implies $\text{vis}[v_j] = v_i$. On the other hand, if $j < i$, then $t(v_i)$ is an ancestor of $t(v_j)$ and so $P(v_k)$ contains $t(v_i)$ and $t(v_j)$. Since G is 2-edge connected, there exists vertex $v_\ell \in N_G(v_i) \cap N_G(v_k)$. Also, $v_\ell \notin S$, for otherwise v_i has two neighbors v_k and v_i in $N_G[v_i] - S$ belonging to the same component at the i-iteration, contradicting the assumption $v_i \in S$. Therefore, in this case v_k has two neighbors v_k and v_ℓ in $N_G[v_k] - S$ such that $\text{vis}[v_k] = \text{vis}[v_\ell] = v_i$. □

Theorem 6 *For a directed path graph G with a tail order provided, Algorithm Directed-Path outputs a maximum realizable set S of G in $O(n+m)$ time.*

Proof. From Theorem 2 and Lemmas 3 and 5, it is straightforward to see the correctness of Algorithm Directed-Path. Furthermore, it is also obvious to see that the algorithm takes $O(n+m)$ time. □

4 Conclusion

In this paper, we present an $O(m\alpha(m,n))$-time algorithm for finding degree-preserving spanning trees on strongly chordal graphs and an $O(m+n)$-time algorithm on directed path graphs, where α is the inverse of Ackermann's function. It is an interesting problem to design an polynomial-time algorithm for finding maximum realizable sets on circular-arc graphs.

References

[1] R. P. Anstee and M. Farber. Characterizations of totally balanced matrices. *Journal of Algorithms*, 5(2):215–230, 1984.

[2] R. Bhatia, S. Khuller, R. Pless, and Y. J. Sussmann. The full-degree spanning tree problem. *Networks*, 36(4):203–209, 2000.

[3] H. Broersma, O. Koppius, H. Tuinstra, A. Huck, T. Kloks, D. Kratsch, and H. Müller. Degree-preserving trees. *Networks*, 35(1):26–39, 2000.

[4] G. Chang and G. Nemhauser. The k-domination and k-stability problems on sun-free chordal graphs. *SIAM J. Algebraic Discrete Methods*, 5:332–345, 1984.

[5] P. Damaschke. Degree-preserving spanning trees in small-degree graphs. *Discrete Math.*, 222(1-3):51–60, 2000.

[6] P. F. Dietz. *Intersection graph algorithms*. PhD thesis, Comp. Sci. Dept., Cornell University, Ithaca, NY, 1984.

[7] M. Farber. Characterizations of strongly chordal graphs. *Discrete Math.*, 43(2-3):173–189, 1983.

[8] M. C. Golumbic. *Algorithmic Graph Theory and Perfect Graphs*. Academic Press, New York, 1980.

[9] A. J. Hoffman, A. W. J. Kolen, and M. Sakarovitch. Totally-balanced and greedy matrices. *SIAM J. Algebraic Discrete Methods*, 6(4):721–730, 1985.

[10] S. Khuller, R. Bhatia, and R. Pless. On local search and placement of meters in networks. *SIAM J. Comput.*, 32(2):470–487 (electronic), 2003.

[11] A. Lubiw. Doubly lexical orderings of matrices. *SIAM J. Comput.*, 16(5):854–879, 1987.

[12] R. Paige and R. E. Tarjan. Tree partition refinement algorithms. *SIAM J. Comput.*, 16:973–989, 1987.

[13] J. P. Spinrad. Doubly lexical ordering of dense 0-1 matrices. *Inform. Process. Lett.*, 45(5):229–235, 1993.

[14] R. E. Tarjan. Efficiency of a good but not linear set union algorithm. *J. ACM*, 22(2):215–225, 1975.